

Distribuirani sistemi

Konzistentnost i replikacija

Razlozi replikacije

* Povećanje pouzdanosti

- Npr. ako je fajl sistem repliciran, moguće je nastaviti sa radom ako neka kopija bude narušena jednostavnim komutiranjem na drugu kopiju.

* Poboljšanje performansi

- U slučaju velike geografske razućenosti DS replikacija je poželjna jer smanjuje vreme pristupa podacima (npr. replicirani Web serveri)

* Replikacija i keširanje su široko prihvaćene tehnike skaliranja (proširljivosti sistema) uz održanje performansi.

- Skalabilnost se obično odražava na performanse sistema.
 - Repliciranje i smeštanje podataka bliže procesima koji ih koriste može poboljšati performanse redukovanjem vremena pristupa, rešavajući problem skalabilnosti.

Problemi uzrokovani replikacijom

* Replikacija dovodi do problema konzistentnosti kopija.

- Da bi kopije bile usaglašene potrebno je izvršiti ažuriranje tako da se privremena nekonzistentnost ne primeti;
- Ažuriranje kopija dovodi do degradacije performansi, naročito u velikim DS jer povećava saobraćaj kroz mrežu.
- Održavanje kopija konzistentnim može ugroziti skalabilnost.

✱ Kopije su konzistentne ako read operacija nad bilo kojom kopijom uvek vraća isti rezultat.

- Kada se obavlja ažuriranje na jednoj kopiji i sve ostale kopije se moraju ažurirati pre nego što se obavi neka druga operacija.
- Sve replike na globalnom nivou treba da se usaglaise kada tačno ažuriranje treba da bude urađeno.
 - Npr. Korišćenjem Lamportovih vremenskih markica ili koordinatora.

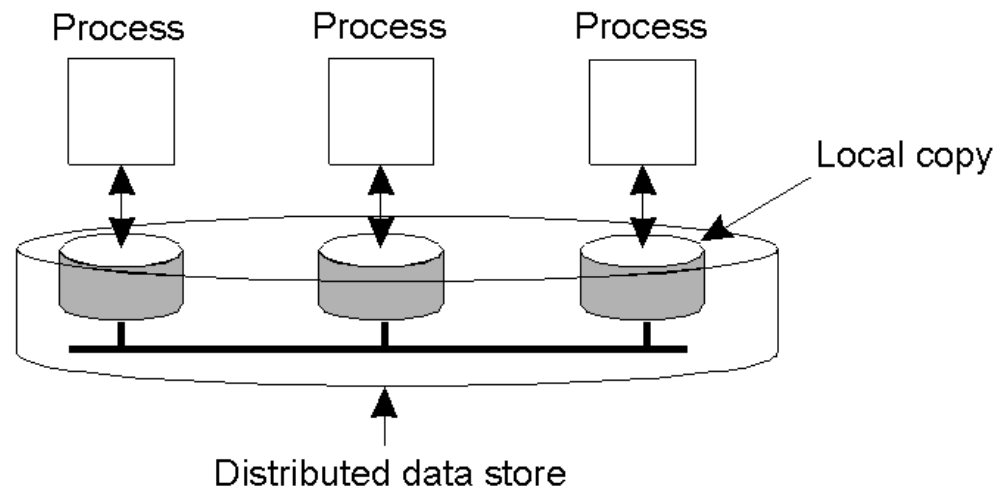
✱ Dilema:

- Problem skalabilnosti može se rešiti korišćenjem replika i keširanja
- Da bi se kopije održale konzistentnim zahteva se globalna sinhronizacija, koja je veoma skupa u pogledu performansi
 - lek može biti opasniji od bolesti!
 - Kada i kako će se izvršiti ažuriranje kopija, određuje cenu replikacije.

Modeli konzistencije

* Konzistencija se uvek razmatra u kontekstu read i write operacija

- Operacije se mogu obavljati nad deljivim podacima koji mogu biti zapamćeni u deljivoj memoriji, deljivoj (distribuiranoj) bazi podataka, ili (distribuiranom) fajl sistemu.
 - Termin skladište podataka (data store) se koristi za ovakve podatke.
 - Skladište podataka može fizički biti distribuirano na više mašina.
 - Usvaja se da svaki proces koji može da pristupi skladištu podataka ima lokalnu (ili u blizini) kopiju celog skladišta.
 - Operacija se označava kao write ako menja podatak, u suprotnom se označava kao read.
 - Write operacije se prenose (propagiraju) drugim kopijama



Modeli konzistencije (nast.)

- * Problem održavanja konzistentnosti distribuiranih skladišta podataka se javlja zbog konačnog vremena potrebnog za distribuciju ažuriranja (komunikaciono kašnjenje) i nepostojanja globalnog časovnika.
- * Konzistencija je definisana u kontekstu read i write operacija nad deljivim podacima
 - Proces koji obavlja read operaciju očekuje da će dobiti vrednost koja predstavlja rezultat poslednje write operacije
 - U odsustvu globalnog časovnika teško je odrediti koja write operacija je bila poslednja.
 - Zbog toga je potrebno dati druge definicije, koje nisu bazirane na poznavanju globalnog vremena, što dovodi do niza modela konzistentnosti.
 - Svaki model definiše vrednosti koje read operacija nad podatcima može vratiti.
 - Koji će se model konzistencije koristiti zavisi od aplikacije

Modeli konzistencije

- * Striktna
- * Sekvencijalna
- * Kauzalna
- * FIFO

Striktna konzistencija

* Najjači model konzistencije:

- *Bilo koja operacija nad podatkom X vraća rezultat poslednje write operacije nad X .*

* Definicija je prirodna i očigledna, ali implicitno usvaja postojanje globalnog vremena, tako da određivanje poslednje write operacije bude nedvosmisleno.

- Jednoprocesorski sistemi tradicionalno podržavaju striktnu konzistenciju.
 - Npr. Ako u programu stoje naredbe $a:=1$; $a:=2$; $\text{print}(a)$, rezultat $\text{print}(a)$ uvek mora biti 2!.
- U sistemu u kome se podaci nalaze na više mašina, kojima može pristupati više procesa, situacija je mnogo složenija:
 - Pretpostavimo da proces p_i ažurira vrednost promenljive x sa 4 na 5 u trenutku t_1 i šalje novu vrednost svim replikama u grupi (multicast)
 - Proces p_j čita (read) vrednost promenljive x u trenutku t_2 ($t_2 > t_1$).
 - Proces p_j treba da pročita vrednost 5, bez obzira kolika je razlika ($t_2 - t_1$).

* U sistemu u kome se podaci nalaze na više mašina, kojima može pristupati više procesa, situacija je mnogo složenija:

- Pretpostavimo da proces P_i ažurira vrednost promenljive x sa 4 na 5 u trenutku t_1 i šalje novu vrednost svim replikama u grupi (multicast)
- Proces P_j čita (read) vrednost promenljive x u trenutku t_2 ($t_2 > t_1$).
- Proces P_j treba da pročita vrednost 5, bez obzira kolika je razlika ($t_2 - t_1$).
- šta ako je $t_2 - t_1 = 1$ nsec i koriste se optička vlakna između dve host mašine (na kojima se izvršavaju ova dva procesa) koje se nalaze na rastojanju od 3 metra?
- Poruka kojom se zahteva ažuriranje morala bi da putuje 10 puta brže od brzine svetlosti
- Po Ajnštajnovoj specijalnoj teoriji relativiteta to nije moguće!

* Zaključak: striktnu konzistenciju u distribuiranom sistemu je nemoguće postići!

* Problem sa striktnom konzistencijom je što se model zasniva na apsolutnom globalnom vremenu.

Striktna konzistencija (nast.)

* Oznake:

- $W_i(x)a$ - označava da proces i obavlja write nad podatkom x , nakon čega je vrednost $x=a$
- $R_i(x)b$ – označava da proces i obavlja read nad X i dobija vrednost b .
 - Svaki podatak je inicijalno postavljen na NIL

P1:	$W(x)a$	
P2:		$R(x)a$

(a)

korektno

P1:	$W(x)a$	
P2:	$R(x)NIL$	$R(x)a$

(b)

Nije korektno sa stanovišta striktne konzistencije.

Striktna konzistencija (nast.)

- * Ako je skladište podataka striktno konzistentno, svi upisi (write) su trenutno vidljivi svim procesima, bez obzira koliko se brzo dešavaju događaji.
- * Ovakav vid konzistentnosti je praktično nemoguće postići u DS.
- * Uvode se slabiji modeli, koji nisu bazirani na apsolutnom vremenu.
 - Iskustva poakzuju da se programeri mogu izboriti sa slabijim modelima konzistencije.
 - U OS i paralelnim sistemima koriste se kritične sekcije i uzajamno isključivanje.
 - Nikakve pretpostavke o brzini izvršenja procesa se ne mogu uvoditi
 - Ako je potrebno procese sinhronizovati, to mora biti ugrađeno u sam program
 - Kritične sekcije omogućavaju da se pristup deljivom resursu linearizuje, tako što će procesi pristupati resursu uzajamno isključivo.
 - Ako je potrebno ostvariti određeni redosled pristupa deljivom resursu to je moguće ostvariti korišćenjem semafora.

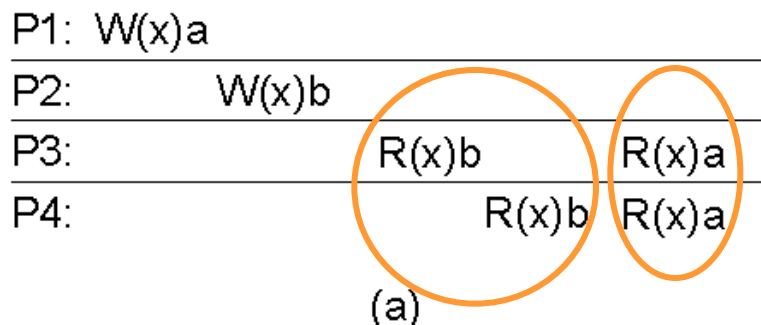
Sekvencijalna konzistencija

* Sekvencijalna konzistencija je slabiji model od striktno:

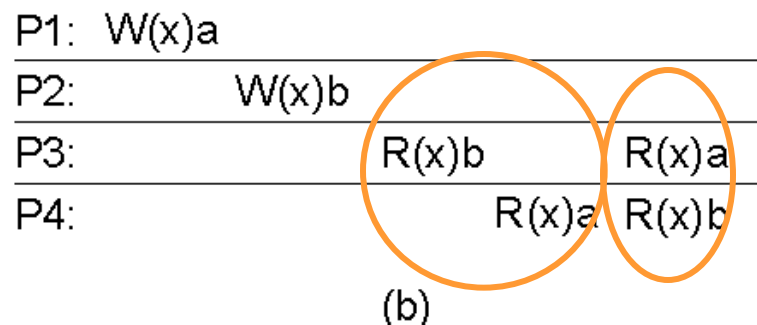
- Rezultat bilo kog izvršenja je isti kao da su (read i write) operacije svih procesa na skladištu podataka izvršene u nekom sekvencijalnom redosledu i operacije svakog pojedinačnog procesa pojavljuju se u ovoj skvenci u redosledu koji je određen njihovim programom.

* Def. kaže da kada se procesi izvršavaju konkurentno na (moguće) različitim mašinama, bilo koje validno preplitanje read i write operacija je prihvatljivo, ali svi procesi vide isto preplitanje operacija.

- Ovo smo videli na primeru bankovnog računa
 - jedna moguća implementacija je pomoću Lamportovih vremenskih markica.
- U definiciji se ne spominje vreme, tj. Nema termina “poslednja write operacija”



Sekvencijalno konzistentno skladište



Narušena sekvencijalna konzistentnost
jer svi procesi ne vide na isti način
operacije.

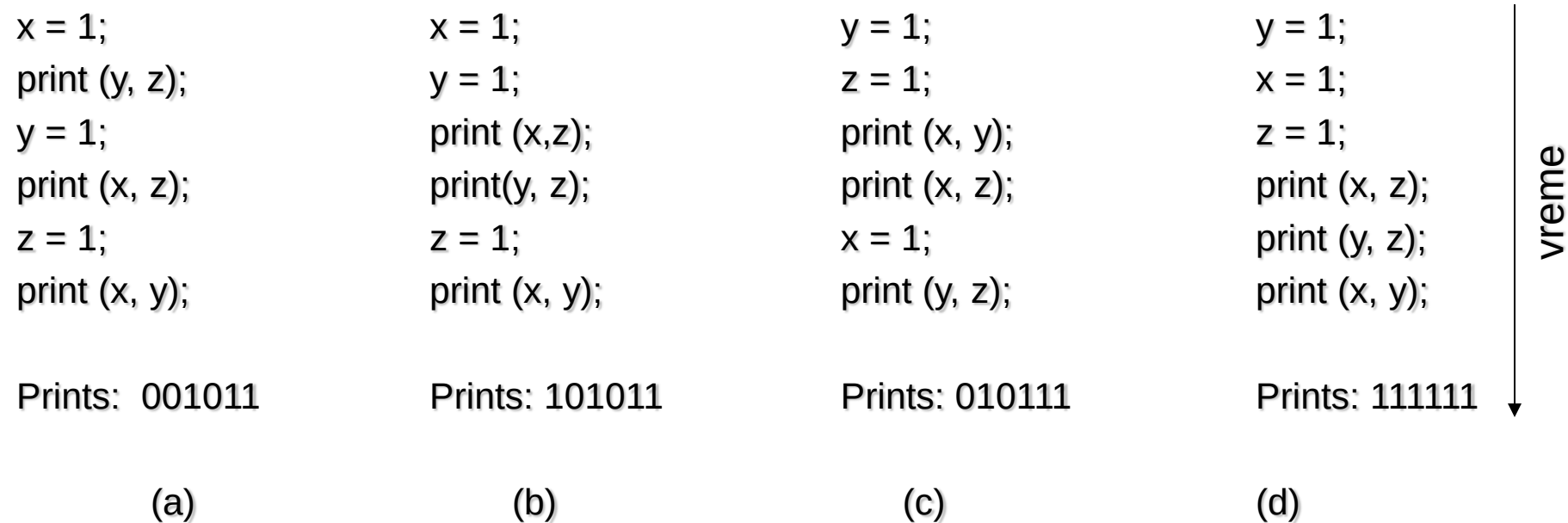
Sekvencijalna konzistencija: Primer

Process P1	Process P2	Process P3	
x = 1;	y = 1;	z = 1;	write
print (y, z);	print (x, z);	print (x, y);	read

* Posmatramo tri procesa koji se konkurentno izvršavaju

- x, y i z su celobrojne promenljive inicijalizovane na 0 i zapamćene u deljivom sekvencijalno konzistentnom skladištu podataka.
- Naredba dodeljivanja odgovara write naredbi, print odgovara read operaciji dva argumenta.
- Moguća su različita preplitanja nizova izvršenja naredbi.
 - Sa 6 naredbi moguće je dobiti 6! (720) različitih preplitanja, mada neka od njih narušavaju programski redosled.
 - Razmotrimo 5! (120) sekvenci koje počinju sa x=1.
 - Pola od njih ima print(x,z) pre y=1 i tako narušava sekvencijalno uređenje programa.
 - Takođe, pola od preostalih permutacija će imati print(x,y) pre z=1 i takođe narušava sekvencijalni redosled.
 - Samo 1/4 od 120 mogućih sekvenci (tj. 30) je validno.
 - Drugih 30 validnih sekvenci moguće je za y=1 na početku, i još 30 za z=1 na početku, tj. postoji 90 validnih sekvenci izvršenja!

Primer (nast.)



Četiri ispravna niza izvršenja naredbi za procese. Vertikalna osa je vreme.

a) Proces se izvršavaju po redosledu P1, P2, P3.

Sledeća tri primera demonstriraju različita, ali validna preplitanje naredbi u vremenu. Svaki od procesa štampa dve promenljive. Jedine vrednosti koje promenljive mogu da uzmu su početna vrednost (0) i dodeljena vrednost 1. svaki proces proizvodi 2-bitni niz. Vrednosti iza prints su izlazi koji se pojavljuju na izlaznom uređaju.

90 različitih validnih sekvenci naredbi generiše različite rezultate (<64) koji su dozvoljeni pod pretpostavkom sekvencijalne konzistencije.

Primer (nast.)

```
x = 1;  
print (y, z);  
y = 1;  
print (x, z);  
z = 1;  
print (x, y);
```

```
x = 1;  
y = 1;  
print (x,z);  
print(y, z);  
z = 1;  
print (x, y);
```

```
y = 1;  
z = 1;  
print (x, y);  
print (x, z);  
x = 1;  
print (y, z);
```

```
y = 1;  
x = 1;  
z = 1;  
print (x, z);  
print (y, z);  
print (x, y);
```

Prints: 001011

Potpis: 001011

(a)

Prints: 101011

101011

(b)

Prints: 010111

110101

(c)

Prints: 111111

111111

(d)

- * ako izlaze iz P1, P2 i P3 navedemo u tom redosledu, dobija se 6-to bitni niz koji karakteriše pojedina preplitanja naredbi (potpis)
- * pošto su moguće vrednosti promenljivih 0 i 1, to znači da ima $2^6=64$ različitih potpisa, ali svi potpisi (tj. preplitanja naredbi nisu dozvoljeni)
- * Npr 000000 nije validan potpis jer bi to značilo da se print naredba izvršila pre naredbe dodeljivanja, narušavajući sekvencijalnu konzistentnost (da se naredbe izvršavaju u redosledu definisanom u svakom od procesa)
- * Šta je sa potpisom 001001?

Primer2

- * Procesi P1, P2 i P3 se konkurentno izvršavaju.
Inicijalno su vrednosti svih promenljivih postavljene na nulu. Nakon okončanja procesa, koje vrednosti promenljivih u, v i w nisu moguće pod uslovima sekvencijalne konzistencije?

<u>P1</u>	<u>P2</u>	<u>P3</u>
A=1	u=A	v=B
B=1	w=A	

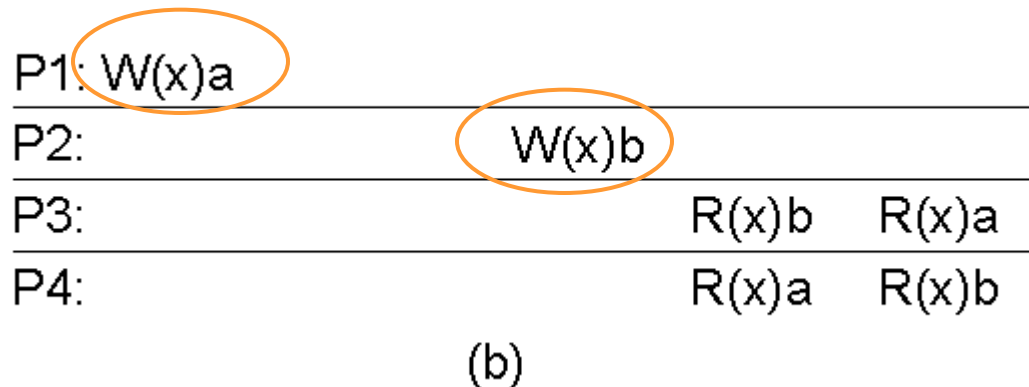
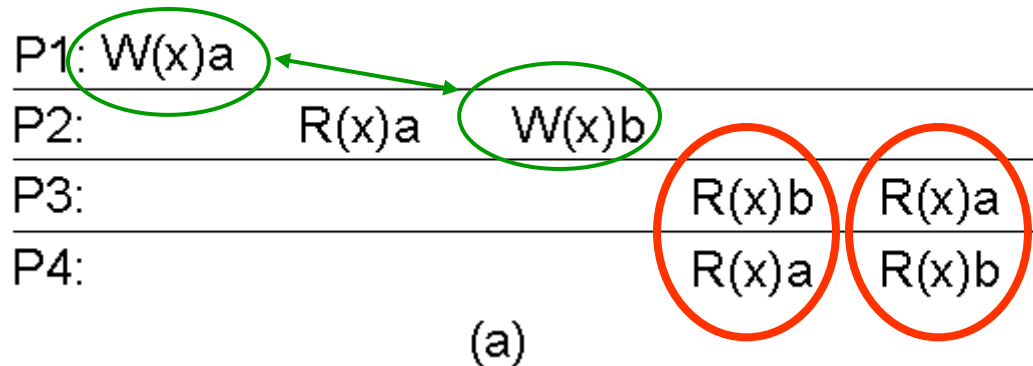
uslovna (kauzalna) konzistencija

- * Upisi (write) koji su potencijalno uslovljeni moraju se videti u svim procesima u istom redosledu
- * Konkurentni upisi se mogu videti u različitom redosledu u različitim procesima

P1:	W(x)a		W(x)c	
P2:		R(x)a	W(x)b	
P3:		R(x)a		R(x)c
P4:		R(x)a		R(x)b

- $W(x)a \rightarrow R(x)a$, $R(x)a \rightarrow W(x)b$, $\Rightarrow W(x)a \rightarrow W(x)b$ (tj. ova dva upisa su potencijalno uslovljena)
- $W(x)c$ i $W(x)b$ nisu potencijalno uslovljeni, pa rezultate ovih upisa različiti procesi mogu videti u različitom redosledu!
- ovakvo skladište podataka je uslovno konzistentno, ali ne i sekvencijalno!

uslovna konzistencija (nast.)



- a) nije ispoštovana uslovna konzistencija
- b) ispoštovana je uslovna konzistencija jer $W(x)a$ i $W(x)b$ nisu medjusobno uslovljeni
- da bi se implementirala uslovna konzistencija mogu se koristiti vektorski časovnici za uredjenje medjusobno zavisnih događaja

FIFO konzistencija

- * Upisi koje obavi jedan proces se vide od strane drugih procesa po redosledu u kome su izdati.
- * ... ali upisi različitih procesa mogu se videti u različitom redosledu u različitim procesima.
- * drugim rečima, ne postoje nikakve garancije o tome kako različiti procesi vide upise drugih procesa, osim što dva ili više upisa iz istog izvora moraju stići po redosledu.
- * Jednostavna implementacija:
 - Svaki proces dodaje poruci ažuriranja : (proces id, redni broj)
 - Svi ostali procesi primenjuju ažuriranja po redosledu u kome su ona obavljena u izvornom procesu.

FIFO konzistencija

P1: $W(x)a$

P2: $R(x)a$ $W(x)b$ $W(x)c$

P3: $R(x)b$ $R(x)a$ $R(x)c$

P4: $R(x)a$ $R(x)b$ $R(x)c$

Ni jedan od prethodnih modela konzistencije ne dozvoljava ovakav redosled dogadjaja

FIFO konzistencija

Process P1	Process P2	Process P3
<code>x = 1;</code> <code>print (y, z);</code>	<code>y = 1;</code> <code>print (x, z);</code>	<code>z = 1;</code> <code>print (x, y);</code> write read

```
x = 1;  
print (y, z);  
y = 1;  
print(x, z);  
z = 1;  
print (x, y);
```

Prints: 00

(a)

```
x = 1;  
y = 1;  
print(x, z);  
print ( y, z);  
z = 1;  
print (x, y);
```

Prints: 10

(b)

```
y = 1;  
print (x, z);  
z = 1;  
print (x, y);  
x = 1;  
print (y, z);
```

Prints: 01

(c)

* naredbe označene masnim slovima generišu prikazane izlaze.

- Konkatencijom izlaza dobija se vrednost 001001, koja nije dozvoljena u slučaju striktno konzistencije.

Sekvencijalna naspram FIFO konzistencije

- * U oba slučaja redosled izvršenja nije determinisan
- * Sekvencijalna: procesi na isti način vide događaje
- * FIFO: procesi mogu videti događaje u različitom redosledu (sem onih koji potiču iz istog procesa)

Process P1	Process P2
<code>x = 1;</code> <code>if (y == 0) kill (P2);</code>	<code>y = 1;</code> <code>if (x == 0) kill (P1);</code>

Pretpostavimo da je inicijalno $x = y = 0$

Sekvencijalna: mogući ishodi: P1 ili P2 ili ni jedna proces nije ubijen

FIFO: moguće je da oba procesa budu ubijena jer različiti procesi mogu videti operacije u različitom redosledu!

Slaba konzistencija

- * Sekvencijalna konzistencija je dosta restriktivna za mnoge aplikacije jer zahteva da sve write operacije jednog procesa budu vidljive svuda u sistemu.
 - Npr. dok se jedan proces nalazi u kritičnoj sekciji i modifikuje podatke u repliciranoj bazi podataka, sve write operacije tog procesa moraju da budu prosleđene svim kopijama, mada dok je proces u kritičnoj sekciji drugi proces ne može pristupiti deljivim podacima.
 - Bolje rešenje je dozvoliti procesu da okonča izvršenje kritične sekcije, a onda obezbediti da konačni rezultat bude prosleđen svima u sistemu.
 - Ovo se može postići korišćenjem eksplicitne sinhronizacije pomoću sinhronizacionih promenljivih.
 - Na sinhronizacionoj promenljivoj, S, se može obavljati samo jedna operacija synchronize (S) , koja sinhronizuje sve lokalne kopije skladišta podataka.
 - Proces P obavlja operacije samo nad svojom lokalnom kopijom skladišta podataka i nema garancija kada će modifikacije biti vidljive drugim procesima.
 - Tek kada se skladište podataka sinhronizuje, svi lokalni upisi procesa P se prosleđuju ostalim kopijama skladišta podataka, a upisi drugih procesa donose u P-ovu kopiju.
 - Model slabe konzistencije ostvaruje konzistenciju nad grupom operacija, a ne nad individualnim read i write operacijama.

Slaba konzistencija

```
int a, b, c, d, e, x, y;  
int *p, *q;  
int f( int *p, int *q);  
prototype */
```

```
a = x * x;  
register */  
b = y * y;  
c = a*a*a + b*b + a * b;  
d = a * a * c;  
p = &a;  
q = &b  
e = f(p, q)
```

```
/* variables */  
/* pointers */  
/* function
```

```
/* a stored in
```

```
/* b as well */
```

```
/* used later */
```

```
/* used later */
```

```
/* p gets address of a */
```

```
/* q gets address of b */
```

```
/* function call */
```

I kod centralizovanih (jednoprocesorskih) sistema postoji privremena nekonzistentnost memorije.